

THIẾT KẾ PHẦN MỀM

SOFTWARE DESIGN

NHẬP MÔN
CÔNG NGHỆ PHẦN MỀM

Vu Son Lam

05/2019

1

Nội dung

1. Một số khái niệm trong thiết kế phần mềm
2. Quy trình thiết kế phần mềm

2

Nội dung

1. Một số khái niệm trong thiết kế phần mềm
2. Quy trình thiết kế phần mềm

3

Một số khái niệm trong thiết kế phần mềm

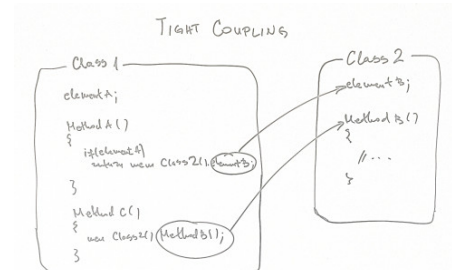
- **Coupling** và **Cohesion** là hai thuộc tính quan trọng trong việc thiết kế phần mềm, nhằm xây dựng một hệ thống dễ bảo trì, dễ sửa đổi.

4

Coupling

- Coupling thể hiện **sự phụ thuộc, sự kết nối** giữa các module với nhau.
- Trong thiết kế, người ta hướng tới mục tiêu **Low coupling**.
- **Low coupling (Loose coupling):**
 - 2 modules được gọi là low coupling nếu chúng **ít phụ thuộc vào nhau**, sự thay đổi trong module này ít làm ảnh hưởng hoặc không làm ảnh hưởng đến module kia.
- **High coupling (Tight coupling):**
 - 2 modules được gọi là high coupling nếu chúng phụ thuộc chặt chẽ vào nhau, sự thay đổi trong module này dẫn đến phải thay đổi ở module kia.

5



Class1 rất phụ thuộc vào Class2 (Khởi tạo Class 2 bên trong Class1)

- Nếu Class2 thay đổi (ví dụ thêm tham số trong phương thức khởi tạo) thì tất cả các lớp có sử dụng Class2 phải thay đổi theo.
- Hoặc lớp Class1 muốn sử dụng một phương thức từ Class3 (thay vì Class2), thì nhiều chỗ trong Class1 phải thay đổi.

6

```

class ClassA {
    private bool _elementA;

    public int MethodA()
    {
        if (_elementA)
            return new ClassB().MethodB();

        return 0;
    }

    public void PrintValues()
    {
        new ClassB().MethodB();
    }
}

```

```

class ClassB {
    public int _elementB;

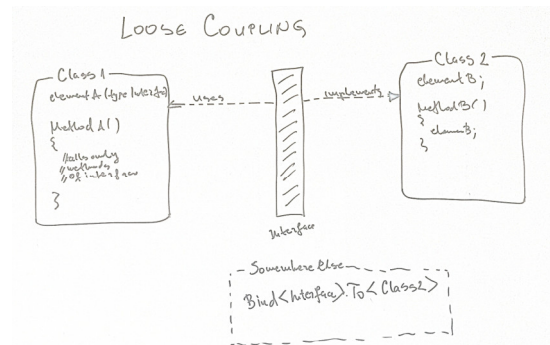
    public void MethodB()
    {
        Console.WriteLine(_elementB);
    }
}

```

7

Giải pháp:

Đảo ngược sự phụ thuộc (Dependency Inversion)



8

```

class ClassA
{
    private readonly ISomeInterface _interfaceImpl;

    public ClassA(ISomeInterface interfaceImpl)
    {
        _interfaceImpl = interfaceImpl;
    }

    public int MethodA()
    {
        return _interfaceImpl.MethodB();
    }

    public void PrintValues()
    {
        _interfaceImpl.PrintValues();
    }
}

```

```

interface ISomeInterface
{
    int MethodB();
    void PrintValues();
}

```

9

```

class ClassB : ISomeInterface
{
    private int _elementB = 2 + 2;

    public int MethodB()
    {
        return _elementB;
    }

    public void PrintValues()
    {
        Console.WriteLine(_elementB);
    }
}

```

```

void main() {
    ISomeInterface obj = new ClassB();

    ClassA objA = new ClassA(obj);

    objA.MethodA();
    objA.PrintValues();
}

```

```

class ClassC : ISomeInterface
{
}

```

```

class ClassD : ISomeInterface
{
}

```

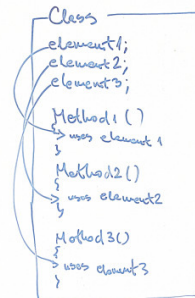
10

Cohesion

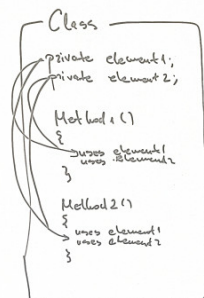
- Cohesion thể hiện sức mạnh **liên kết** giữa các thành phần trong cùng một module.
- Khi thiết kế module, cần hướng tới **High cohesion**.
- High cohesion:**
 - Một module được gọi là high cohesion nếu các thành phần bên trong nó **có liên kết chặt chẽ với nhau**.
 - Nếu xét một module là class, thì các thành phần bên trong nó sẽ là các trường dữ liệu, các phương thức.
- Low cohesion:**
 - Nếu các thành phần bên trong nó **rời rạc, không liên kết với nhau**.
 - Một module mà thực hiện nhiều chức năng khác nhau, sẽ dễ dẫn tới low cohesion.
 - Ví dụ lớp Utility: định dạng ngày giờ, định dạng đường dẫn...

11

Low Cohesion



High Cohesion



12

Cohesion

- Có nhiều cách để chia hệ thống thành các module nhỏ khác nhau.
- Việc phân chia tốt sẽ làm cho hệ thống đạt được tính chất **high cohesion** trong mỗi module và tính chất **low coupling** giữa các module.
- Xét trường hợp module là class, khi thiết kế phải trả lời các câu hỏi:
 - Nên có **bao nhiêu lớp**? Là **những lớp nào**?
 - Trong lớp đó, sẽ định nghĩa **những phương thức gì**, **lưu dữ liệu gì**?

13

Các loại cohesion

- Một số kiểu kết hợp, tạo nên module có các loại cohesion khác nhau. Dưới đây là các loại được tổng hợp, từ tệ nhất đến tốt nhất:
 - Coincidental cohesion (worst)
 - Logical cohesion
 - Temporal cohesion
 - Procedural cohesion
 - Communicational/informational cohesion
 - Sequential cohesion
 - Functional cohesion (best)

14

Các loại cohesion Functional cohesion

- Functional cohesion là kiểu kết hợp mà các thành phần trong một module được kết hợp với nhau theo tiêu chí **cùng thực hiện một chức năng chung duy nhất** của module.
- Nghĩa là module đó (class đó) phải được thiết kế để chỉ làm một nhiệm vụ duy nhất (không nên làm nhiều chức năng như lớp Utility)
- Ví dụ:
 - Class **ProductDAO**: truy xuất dữ liệu cho **Sản phẩm** (không đưa vào **GetUserInfo**, **DisplayGridView**...)

15

Nội dung

1. Một số khái niệm trong thiết kế phần mềm
2. Quy trình thiết kế phần mềm

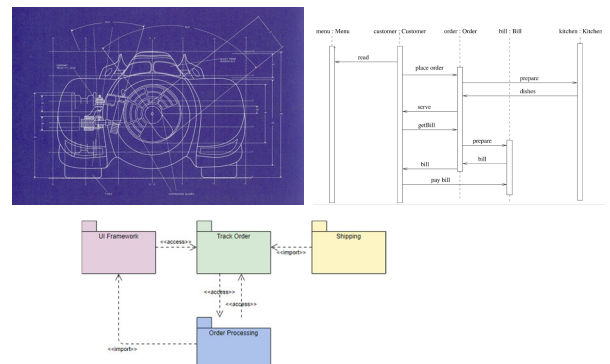
16

Quy trình thiết kế phần mềm

- The design activity begins when the **requirements** document for the software to be developed is **available** and the **architecture** has been **designed**.
- During design we further **refine** the **architecture**.
- Generally, design focuses on what we have called the **module view**.
- During design we determine **what modules the system should have** and which have to be developed.
- Often, the module view may effectively be a module structure of each component in the architecture.

17

The design of a system is essentially a **blueprint** or a plan for a solution for the system.



18

Quy trình thiết kế phần mềm

- The design process for software systems often has two levels:
 - At the first level the focus is on deciding:
 - which modules are needed for the system?
 - specifications of these modules?
 - how the modules should be interconnected?
 - This is what may be called the **module design** or the **high-level design**.

19

Quy trình thiết kế phần mềm

- The design process for software systems (cont.):
 - In the second level, the **internal design of the modules**, or **how the specifications of the module can be satisfied**, is decided.
 - This design level is often called **detailed design** or **logic design**.
 - Detailed design essentially expands the system design to contain a **more detailed description of the processing logic and data structures** so that the design is sufficiently complete for coding.

20

Tham khảo

[1] Ian Sommerville, *Software Engineering*, 9th ed., Addison-Wasley, 2011.

[2] Pankaj Jalote, *A Concise Introduction to Software Engineering*, Springer, 2008.

Website:

- TehBoyan, <https://thebojan.ninja/2015/04/08/high-cohesion-loose-coupling/>
- <https://edwardthienhoang.wordpress.com>

21